Requirements Analysis and Prototyping using Scenarios and Statecharts

Henrik Behrens
FernUniversität Hagen
58084 Hagen
Germany
+49 2331 987 2966
Henrik.Behrens@fernuni-hagen.de

ABSTRACT
This paper shows a way to model the requirements of a business information system using scenarios and statecharts. Inconsistency and redundancy problems are avoided by generating the scenarios from an integrated structured state chart model. The approach uses precise action semantics, supports changing requirements and enables seamless generation of a fully functional prototype for end user requirements validation. The method is currently being implemented in the STAMP tool (State Modeling and Prototyping).

Categories and Subject Descriptors
D.2.1 [**Software Engineering**]: Requirements/Specifications; D.2.2 [**Software Engineering**]: Design Tools and Techniques – *State Diagrams.*

General Terms
Management, Design, Languages.

Keywords
Requirements Specification, Scenarios, Statecharts, Change Management, ASL, Prototype, Visualization.

1. INTRODUCTION
Capturing the functional requirements of a software system usually involves creating structural and behavioral models. For behavioral modeling, UML offers interaction and statechart diagrams, allowing scenario-based and state-based modeling.

We define a scenario to be a sequential series of user and/or system actions that might reasonably take place.

Scenarios have become key artifacts in systems engineering, but their management is poorly understood [7]. Scenarios are a good communication base with customers and other non-technical people and support early validation of requirements at a low abstraction level [9]. However, the full potential of scenarios is not always fully exploited because of problems including the following [9][14]:

- modeling the behavior of a whole system with scenarios requires a great multitude of scenarios
- scenarios are highly redundant, some scenario parts are part of many scenarios, sometimes with small variations
- misunderstandings happen because of ambiguities
- missing traceability between scenarios and other software artifacts
- UML interaction diagrams (which are often used to describe scenarios) lack adequate expressiveness and semantic foundation
- weak visualization techniques
- lack of change management
- insufficient tool support

The first two problems can be addressed by generating a statechart model from the scenarios [2][15][16][20]. The scenarios are incorporated and merged in the statechart model and can then be generated back from the statechart using a statechart driver [6][16]. The approach of generating statecharts from scenarios contains the following problems, which need to be solved [2][16][20]:

- Scenarios can be inconsistent (contradictory), these inconsistencies must be detected and resolved.
- In order to avoid redundancy, common parts of the scenarios must be detected and merged.
- The resulting state machines might be "overgeneralized", accepting unwanted or erroneous paths.

Our way of modeling requirements avoids these problems by generating scenarios from statecharts. In addition, it has the following useful properties:

- Scenarios and state machines are executable at any time, allowing animated simulations with dynamic visualization of the system's domain object structure.
- Informal and formal action specification elements provide great expressiveness and unambiguous action semantics.
- The approach allows seamless transition to a fully functional prototype for end user validation (including a graphical user interface and working functionality, but not including technical aspects such as persistence, network collaboration, undo-mechanisms and so on).
- The approach explicitly supports changing requirements.

In chapter 2 we classify our approach according to the four scenario views identified by Jarke (content, purpose, form, and life-cycle [8]). Chapter 3 introduces our concept for scenario management, while chapter 4 shows how to deal with changing

requirements. Chapter 5 demonstrates how to specify action semantics unambiguously. In chapter 6 and 7 we discuss scalability aspects and prototype generation, respectively. The paper concludes with a short summary (chapter 8).

2. SCENARIO-BASED DESIGN

2.1 Content

Using scenarios, we model *interactions* (between the user and the system), *system internal actions* (as far as they are non-technical but domain-related) and *contextual actions* (actions performed outside the system), as far as they are necessary to understand the scenarios.

In our opinion, a software system (e.g. a business application) can often not be adequately specified by modeling user interactions only, because internal actions (such as operations modifying domain objects) can be important for a scenario even if their effect is not immediately visible during that user interaction. As an example, registering a client might include the creation of an account object that will later be used to record financial transactions with this client (see section 5.2). We explicitly model the impact of user interactions on the system's data. Our opinion is supported by results of an evaluation of scenario notations by Amyot/Eberlein [1], who state that descriptions of activities performed internally by the system can be of tremendous help for requirements analysis.

It is possible to describe system internal actions using UML sequence or collaboration diagrams or (if combined with statecharts) using story-charts [11]. This implies that every action of the scenario must be associated with explicitly specified sender and receiver objects. We agree with Amyot/Eberlein in the view that such information often belongs to detailed design and should not be part of a requirements model.

2.2 Purpose

The scenarios are used to model system requirements as a basis for prototype generation and implementation. Additionally they are used for validation by analysts, testers, and end users.

2.3 Form

For describing the semantics of scenarios, we use both informal and formal notations. Informal notations are good for recording requirements at an early stage, when great expressiveness and ease of use is more important than formal correctness and executability. An industrial-scale modeling language should allow its users to adapt the degree of formalism in a specification to the difficulty and the risk of the problem at hand [4].

On the other hand, sometimes "analysis" gets confused with "vagueness" leading to models that cannot be understood without significant interpretation by the system designers [10]. Therefore we use precise action specifications (in addition to natural language action descriptions) to capture requirements in an unambiguous and exact way and to enable the seamless generation of a fully functional prototype, which can be very valuable for requirements validation by the end user. We use the ASL action specification language [21] (which is conform to the emerging UML action language specification [22]) to specify action semantics and guard conditions formally and thereby make scenarios executable. Further details will be discussed in chapter 5.

2.4 Life cycle

In chapter 1, we mentioned the problem that modeling the behavior of a whole system with scenarios requires a great multitude of scenarios. If a system has, for example, 50 use cases and 20 important scenarios per use case, it would be necessary to keep 1000 scenarios consistent (usually under changing requirements). Instead, we model a statechart diagram for each use case which can generate all scenarios for that use case. A scenario is represented in the statechart diagram as a path (consisting of state nodes and transition edges) through the statechart diagram. It is possible to use action diagrams instead of statechart diagrams [13], but statechart diagrams have more notational options, allowing to attach entry-, doActivity- and exit-actions to states and transition actions to state transitions.

3. GENERATING SCENARIOS FROM STATE MACHINES

3.1 Concept

The idea of using a statechart for capturing scenarios is favored by several authors, e.g. [2][12][15][16][19][20]. They differ in the view about how the scenarios and the statechart model should be generated from each other and how the problems mentioned in chapter 1 (detection of inconsistent scenarios, detecting common parts for merging and solving the problem of an "overgeneralized" state machine) should be handled.

According to Whittle, inconsistent scenarios arise due to the fact that they are usually written in isolation. To enable automatic detection of conflicts, he advises the analyst to give OCL constraints [20].

We suggest to solve the problem by avoiding to write scenarios in isolation. Instead, we write a scenario by adding a few states and state transitions to the state machine of the use case which the scenario belongs to, such that the requested scenario becomes executable as a path through the statechart diagram.

We agree to the view of Koskimies/Systä that scenarios and statecharts should be developed in concert rather than sequentially [18]. The difference between their and our approach is the preferred direction of generation: while they typically write scenarios and generate statecharts, we typically write state charts (or supplement them, which is usually less work and less redundant than writing whole scenarios) and generate scenarios. This way we avoid to produce redundant models from the beginning (scenarios are highly redundant) and avoid conflicts between inconsistent scenarios.

As we prefer to work with *concrete scenarios* rather than *abstract scenarios* (*design by example*, [12]) we have to provide concrete data values for each scenario being incorporated into a state machine. Again, we try to enter as little data as possible to avoid redundancy. To generate a scenario from a statechart, the following additional information is needed:

- user actions triggering state transitions (event occurrences, e.g. pressing a button or selecting a menu item)
- user actions providing data input (e.g. filling out a form or selecting a data item)
- the object constellation that represents the state of the whole system at the beginning of scenario execution

The object constellation in turn depends on user actions of previously executed scenarios that caused this constellation to be created. Therefore, if subsequent execution of scenarios (*sequential scenario composition*) is supported, it is sufficient to store the above mentioned user actions and the state diagram in order to be able to reproduce all scenarios.

3.2 Example

Consider the domain class diagram of a system called SeminarIS (Seminar Information System) in figure 1 and the state diagram for the use case "enroll a customer for a seminar" in figure 2. For easy reference, we numbered all states and state transitions. The scenario "Create a customer named Smith and a seminar titled UML and enroll him for this seminar" can be specified as a path through the statechart, supplemented with some user input for each state or transition containing the (UI) symbol, denoting user input (figure 3). Figure 4 shows the object constellation of the SeminarIS system after the execution of this scenario.

In figure 2, we could have replaced the state "select customer and seminar" with the states "nothing selected", "customer selected", "seminar selected" and "customer and seminar selected", but then the outgoing transitions numbered 2, 3, 4, and 7 would have to be outgoing transitions of all four states, causing redundancy. Alternatively, they could have been outgoing transitions of a *composite state* containing the above four states or containing two *concurrent transitions*, one for selecting the customer and the other for selecting the seminar, but that would make the model more complex. Instead, we model the situation that customers and seminars can be selected (and entered, if necessary), as one state.

Note that the state diagram in figure 2 is informal and incomplete. For example, it does not show how the attributes of newly created objects must be initialized and that creating an Enrollment object requires creating an AccountingEntry object and a link between the two in order to satisfy the multiplicity constraints in the class diagram in Figure 1. In order to specify actions precise enough, we use action language statements (see chapter 5).

3.3 Tool support

The STAMP tool allows the analyst to capture a scenario interactively, traversing the state diagram beginning at the initial state, adding states and state transitions with the associated actions for the scenario as necessary. After a scenario step is added, STAMP automatically executes the actions associated with the new step, which causes the object diagram showing the current object constellation to be updated. This way the analyst can watch the object diagram in figure 4 evolve while he enters the scenario in figure 3 (design by animation [12]).

Visualizing the system's objects dynamically during scenario capture and replay helps the analyst to understand the actions performed during scenario execution. Displaying an object model in addition to a class model has significant benefit because class models are inappropriate when more than one object of the same class and/or collaborations between objects have to be modeled [16].

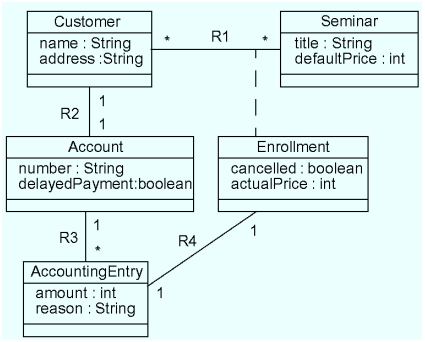

Figure 1. Class Diagram of the SeminarIS system

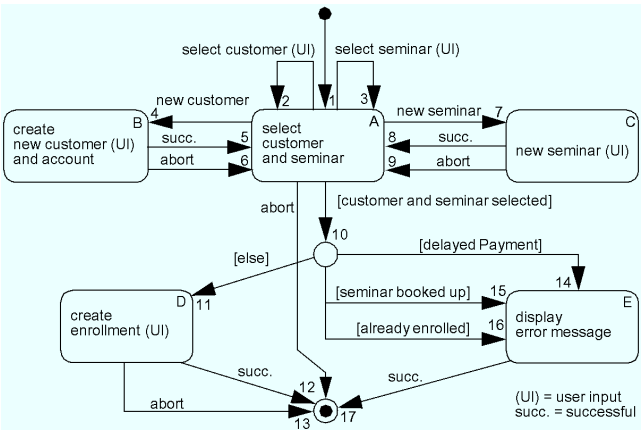

Figure 2. State diagram for the use-case "enroll customer"

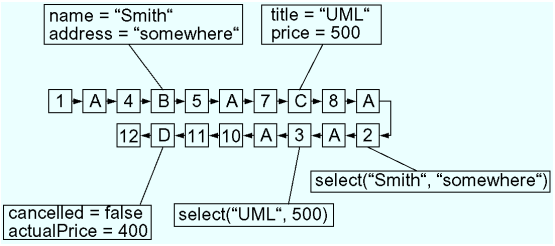

Figure 3. A scenario specification described as path though the statechart diagram in figure 2.

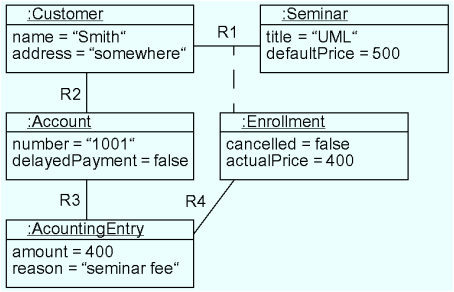

Figure 4. The SeminarIS object diagram after execution of the scenario in figure 3.

4. DEALING WITH CHANGING REQUIREMENTS

4.1 How to Apply Changes to the Model

Dealing with changing requirements is one of the most difficult, yet important challenges for any requirements modeling method, especially for methods using scenarios to capture requirements. Since scenarios are often redundant, a change in the requirements usually affects several scenarios. The redundancy is eliminated in the state model representation of the scenarios, therefore a change in requirements will typically cause less changes in the state model compared to the number of changes necessary in the corresponding scenarios. To apply a change of the requirements into our scenario model, we modify the state model so that the scenarios generated from it have the required behavior.

4.1.1 How to Add New Functionality

To add new functionality to the model, we add additional transitions between existing states or create new states if necessary. If a state has more than one outgoing transition, they must be triggered by different event types, or, if triggered by the same event type, they must have excluding guard conditions to ensure deterministic scenario execution.

If a new transition is triggered by a new event type (i.e. an event type not used in the state diagram before), all existing scenarios are not affected by this change. Likewise, if the guard condition of all new transitions is never evaluated to true in all existing scenarios, they remain unaffected by the change.

4.1.2 How to Remove Existing Functionality

Existing functionality can be removed by removing transitions and/or states. To avoid undefined behavior (e.g. because of states with no outgoing transitions) it may be necessary to add new transitions to fill out the hole.

If some functionality associated with a certain state must be removed from some scenarios traversing this state and must remain in other scenarios traversing this state (this can happen if a special case is detected in the requirements) it is possible to add a transition that skips this state. The guard conditions must be chosen in a way that the state is skipped exactly for the scenarios which should not traverse this state anymore.

4.2 Impact of State Machine Modifications on Scenarios

4.2.1 Problems caused by State Machine Modifications

After each modification of the state model, the scenarios are regenerated from it by executing the actions associated with the states and transitions using the user input values for each scenario. (Each scenario starts with the initial state and ends in the final state.) By the regeneration process, modifications of the state model are propagated to the scenario model automatically. The resulting changes in the scenario model may cause two different sorts of problems, that need further attention of the analyst:

- Improper modifications to the state model might have had unwanted effects on some scenarios. To discover this, the change impact on the scenarios must be visualized immediately. The analyst can then choose to make further modifications to the state model (in order to correct it) or choose to undo the last modification.
- If states are added to the state model which require user input, the analyst may choose to enter different input values for each scenario or allow the system to use some default values for all scenarios.
- Modifications of the state model may cause scenarios to follow a different path in the state diagram than before, causing the associated scenarios to behave very differently. This is rarely wanted, but if it is, and if the traversed states require user input, then again the analyst must provide the necessary user input or let the system use default values instead.

4.2.2 Scenario Change Flags

To give an overview over the effect of a state model modification on the scenarios, the STAMP tool compares the newly chosen path of every scenario together with the new resulting object constellation with the path and the resulting object constellation of the scenarios prior to the change. In this process, the following types of scenario changes are automatically detected:

- *result change*: the object constellation resulting from scenario execution is different than the result prior to the modification
- *path change*: the scenario runs a different path through the state diagram than before
- *user input change*: the user input required to run the scenario has changed

After a modification of the state model, the STAMP tool marks all affected scenarios with the above flags. The analyst watches the change impact on the scenarios and either accepts the changes (by resetting the flags and optionally providing user input values) or revokes the modification. If the analyst does not care about the concrete user input values, he can instruct the system to always use default values for user input.

5. ACTION SPECIFICATION

5.1 Operational vs. Declarative Specification

In the UML context, the semantics of actions can be specified in one of two ways:

- declarative, e.g. using OCL pre- and postconditions
- operational, e.g. using action specification language statements

Declarative specifications are good for testing and verifying, because pre- and postconditions can be tested and used to prove correctness. Declarative specifications are not particularly suitable to create executable models, because the execution of OCL expressions is ambiguous [17].

An *operational specification* uses *statements* or *messages* to describe semantics in a precise way. The ASL (Action Specification Language [21]) is designed to fill the semantic gap in UML and conforms to the UML Action Semantics Request for Proposal issued by the OMG [22]. It supports platform-independently executable action specifications for UML models and provides easy-to-understand operations on UML classes, objects and associations (which is not the case in usual programming languages). For details, see [21].

5.2 Example

The scenario in figure 3 would cause the following ASL statements to be executed (in addition to some guards not included here), creating the object constellation displayed in figure 4:

```
newCustomer = create Customer with name="Smith" and address="somewhere";
newAccount = create Account with number="1001" and delayedPayment = false;
link newCustomer R2 newAccount;
newSeminar = create Seminar with title="UML" and defaultPrice=500;
newEnrollment = create Enrollment with cancelled=false and actualPrice=400;
link newCustomer and newSeminar R1 newEnrollment.
newAEntry = create AccountingEntry with amount=400 and reason="seminar fee";
link newAccount R3 newAEntry;
link newAEntry R4 newEnrollment;
```

The visibility scope of the variable names used in the ASL statements is the associated use case. For data transfer into other visibility scopes, ASL uses so-called "bridge mappings" [10].

6. INTRODUCING STRUCTURE FOR SCALABILITY

6.1 Use-Case Partitioning

If a model for a system must scale up to big applications, some kind of structure has to be introduced to keep the model manageable. In section 2.4, we mentioned that a requirements analysis may well include 1000 different scenarios. As it is difficult to maintain a single state diagram realizing 1000 scenarios, it is necessary to introduce some structure for state diagrams. Many authors agree that the use of hierarchy is crucial to a good design of state machines, e.g. [2][20].

However, opinions differ on *how* a good design should be archived, i.e. which criterion should be used for partitioning the state model and how to apply this partitioning criterion to it.

Whittle [20] recursively partitions the set of state nodes according to the different values of the variables in so-called *state vectors*. Since this partitioning is not unique, he introduces additional heuristic constraints on the layout of the state chart to rule out unreadable partitions. He also draws partitioning criterions from the class diagram (subclass hierarchy and/or aggregation hierarchies).

We agree with Bordelau and Corriveau that automation is not the right approach for creating a good design. They derive the partitioning from different scenario relationships (interaction, dependency and clustering [3]) which each infer a different partitioning for the resulting state model. Their dependency relationship between scenarios resembles the «include» and «extend» dependencies defined in the UML 1.3 [23].

We follow Kösters/Six/Winter choosing use cases as partitioning criterion for statecharts. The «include» and «extend» dependencies between use-cases introduce dependency relationships between the associated statecharts [13]. Figure 5 shows how they are realized in a straightforward way by adding a *composite state* [23] for the included or extending use case in the state diagram of the including or extended use case. This introduces a hierarchical structure on the state diagrams, although the hierarchy is not always strict, as a use-case can be included by more than one other use-case. However, this possibility is important to prevent redundancy and to enable reuse of *subscenarios* [12][15] in other scenarios.

6.2 Subscenarios and application states

A subscenario is a part of a scenario which is realized by one state diagram (possibly containing other substate diagrams). It runs from the initial to the final state of this diagram, but not from the initial to the final state of the whole system state diagram. Subscenarios can be executed one after the other (*sequential scenario composition*) to form bigger scenarios.

Execution of a subscenario transforms the system from one *application state* into another. *Application states* and the *transitions performed by subscenarios* form an *application state diagram* describing possible uses of a system. The application state diagram has nothing to do with the *use case state diagrams* (modeling the system behavior) we discussed in the previous chapters. In an application state diagram, a state is defined by a specific object constellation of the system, while the state of a use case state diagram is defined by the set of possible interactions (corresponding to outgoing transitions) at a specific stage of use case execution. A complete system test (covering all relevant scenarios) could be defined by the set of all paths through the application state diagram.

In general, use-case partitioning satisfies the criteria of low coupling and high cohesion better than partitioning strategies based on class or object diagrams, because it is more likely that a subscenario fits in one use case than that it fits in one class or object.

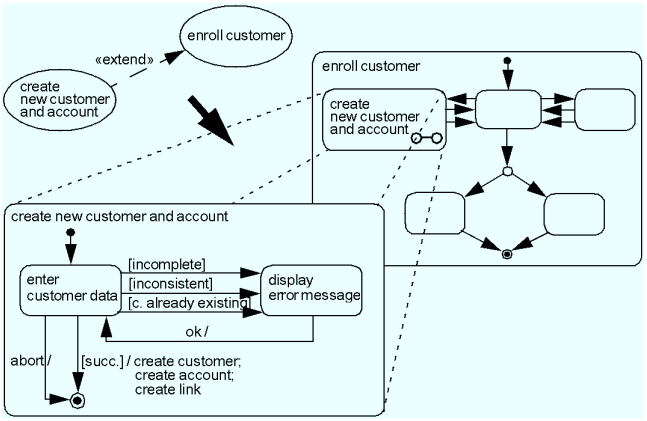

Figure 5. Mapping use case dependencies to statechart dependencies

7. GENERATION OF A PROTOTYPE WITH GRAPHICAL USER INTERFACE

The STAMP tool allows dynamic scenario execution without a graphical user interface being designed, thereby speeding up the validation process. However, often end users would prefer to validate the requirements using a prototype with a user interface which is similar to that of the final product. Therefore, the STAMP tool allows integration of user interface specifications into the state models. This is possible in a straightforward way because we identify states according to the set of possible user interactions in that state. The additional advantage is that there is often a 1:1-correspondence between states and the currently active window (or scene, as the platform-independent counterpart is called in the FLUID approach [5]). When a state is entered, a signal is sent to the user interface causing it to prepare for the possible interactions in that state. From the FLUID approach, we

adopt a high-level class framework for user interface specification, which allows automatic prototype generation for different platforms (window-based, web-based, WAP-based and others). Further details exceed the scope of this paper.

8. SUMMARY

The STAMP approach to requirements specification allows an analyst to build an integrated requirements model with informal and formal action specifications and hierarchical decomposition. By generating scenarios from statecharts, it avoids inconsistency and redundancy problems, which are typical for scenario-based approaches. It explicitly supports changing requirements and allows immediate visualization of the effect of state model modifications on the scenario model. It allows scenario simulation, dynamically visualizing the object model after each scenario execution step. Furthermore, it supports generation of a fully functional prototype with a graphical user interface.

9. REFERENCES

[1] D. Amyot, and A. Eberlein. An Evaluation of Scenario Notations for Telecommunication Systems Development. Proc. 9th International Conference on Telecommunication Systems, Dallas, March 2001.

[2] F. Bordeleau, J.-P. Corriveau. From Scenarios to Hierarchical State Machines: A Pattern based Approach. Proceedings of OOPSLA 2000 Workshop: Scenario based round-trip engineering, October 2000.

[3] F. Bordeleau, J.-P. Corriveau, and B. Selic. A Scenario-Based Approach to Hierarchical State Machine Design. Proceedings of the Third IEEE International Symposium on Object-Oriented Real-Time Distributed Computing, 1998.

[4] M. Glinz et al. The ADORA Approach to Object-Oriented Modeling of Software. In Dittrich et al.: Advanced Information Systems Engineering, Proceedings of CAiSE 2001, Interlaken, Switzerland. Notes in Computer Science Vol. 2068. Berlin: Springer. 76-92.

[5] A. Homrighausen and J. Voss. Tool support for the model-based development of interactive applications - The FLUID approach. Proc. 4th Eurographics Workshop on Design, Specification and Verification of Interactive Systems, Granada, 1997.

[6] P. Hsia et al. Formal Approach to Scenario Analysis. IEEE Software Vol. 11 No. 2 pp. 33-41 March 1994.

[7] M. Jarke. Scenarios for Modelling. Communications of the ACM, Volume 42, 1999.

[8] M. Jarke. CREWS: Towards Systematic Usage of Scenarios, Use Cases and Scenes, Proc. 4. Internationale Tagung Wirtschaftsinformatik (WI'99), Saarbrücken, 469-486 (1999)

[9] M. Jarke, K. Pohl et al. Scenario Use in European Software Organizations - Results from Site Visits and Questionnaires. CREWS-Report 97-10, Namur, Aachen 1997.

[10] Kennedy Carter Ltd. Supporting Model Driven Architecture with eXecutable UML, http://www.kc.com/MDA/xuml.html, 2001.

[11] H. Köhler, U. Nickel, J. Niere, and A. Zündorf. Integrating UML Diagrams for Production Control Systems; in Proc. of ICSE 2000 - Limerick, Ireland, acm press, pp. 241-251, 2000.

[12] K. Koskimies, T. Systä, J. Tuomi, and T. Männistö. Automated Support for Modeling OO Software. IEEE Software, 15(1): 87-94, 1998.

[13] G. Kösters, H.-W. Six, and M. Winter. Coupling Use Cases and Class Models as a Means for Validation and Verification of Requirements Specifications. Requirements Engineering, Vol. 6 No. 1 pp. 3-17, Springer, London, January 2001.

[14] I. Krüger. Notational and Methodical Issues in Forward Engineering with MSCs. Proceedings of OOPSLA 2000 Workshop: Scenario based round-trip engineering, Tampere University of Technology, Software Systems Laboratory, Report 20, October 2000.

[15] S. Leue, L. Mehrmann, and M. Rezai. Synthesizing ROOM Models From Message Sequence Charts Specifications. Proc. 13th IEEE Conf. on Automated Software Engineering, 1998.

[16] E. Mäkinen and T. Systä. An Interactive Approach for Synthesizing UML Statechart Diagrams from Sequence Diagrams. Proceedings of OOPSLA 2000 Workshop: Scenario based round-trip engineering, October 2000.

[17] I. Oliver. Executing the OCL, Proceedings of the ECOOP'99 Workshop for PhD Students in OO Systems (PhDOOS '99), Lisbon, Portugal, 1999.

[18] T. Systä. Incremental Construction of Dynamic Models for Object-Oriented Software Systems. Journal of Object-Oriented Programming Vol. 13 No. 5 pp. 18-27, September 2000.

[19] S. Uchitel and J. Kramer. A Workbench for Synthesizing Behavior Models from Scenarios. Proc. of the 23rd IEEE International Conference on Software Engineering (ICSE'01). Toronto, Canada. May 2001.

[20] J. Whittle and J. Schumann. Generating Statechart Designs From Scenarios. Proceedings of OOPSLA 2000 Workshop: Scenario based round-trip engineering, Tampere University of Technology, Software Systems Laboratory, Report 20, October 2000.

[21] I. Wilkie et al. UML Action Specification Language (ASL) Reference Guide, http://www.kc.com/download/index.html, 2001.

[22] http://www.umlactionsemantics.org

[23] UML 1.3 Specification (final release) http://www.jeckle.de/files/omg-uml-1.3.pdf